Train a Model

For an overview on training a model with Amazon SageMaker, see Train a Model with Amazon SageMaker (p. 4).

Amazon SageMaker provides features to monitor and manage the training and validation of machine learning models. For guidance on metrics available, incremental training, automatic model tuning, and the use of augmented manifest files to label training data, see the following topics.

- For guidance on metrics used to monitor and train models, see Monitor and Analyze Training Jobs Using Metrics (p. 276).
- For guidance on incremental training in Amazon SageMaker, see Incremental Training in Amazon SageMaker (p. 282).
- For guidance on using managed spot training in Amazon SageMaker, see Managed Spot Training in Amazon SageMaker (p. 287).
- For guidance on using training checkpoints in Amazon SageMaker, see Using Checkpoints in Amazon SageMaker (p. 288).
- For guidance on automatic model tuning, also known as hyperparameter tuning, see Automatic Model Tuning (p. 288).
- For guidance on using an augmented manifest file to label training data, see Provide Dataset Metadata to Training Jobs with an Augmented Manifest File (p. 308).

Monitor and Analyze Training Jobs Using Metrics

An Amazon SageMaker training job is an iterative process that teaches a model to make predictions by presenting examples from a training dataset. Typically, a training algorithm computes several metrics, such as training error and prediction accuracy. These metrics help diagnose whether the model is learning well and will generalize well for making predictions on unseen data. The training algorithm writes the values of these metrics to logs, which Amazon SageMaker monitors and sends to Amazon CloudWatch in real time. To analyze the performance of your training job, you can view graphs of these metrics in CloudWatch. When a training job has completed, you can also get a list of the metric values that it computes in its final iteration by calling the DescribeTrainingJob (p. 744) operation.

Topics

- Training Metrics Sample Notebooks (p. 276)
- Defining Training Metrics (p. 277)
- Monitoring Training Job Metrics (Console) (p. 279)
- Monitoring Training Job Metrics (Amazon SageMaker Console) (p. 279)
- Example: Viewing a Training and Validation Curve (p. 281)

Training Metrics Sample Notebooks

The following sample notebooks show how to view and plot training metrics:

- An Introduction to the Amazon SageMaker ObjectToVec Model for Sequence-to-sequence Embedding (object2vec_sentence_similarity.ipynb)

- Regression with the Amazon SageMaker XGBoost Algorithm (xgboost_abalone.ipynb)

For instructions how to create and access Jupyter notebook instances that you can use to run the examples in Amazon SageMaker, see Use Example Notebooks (p. 42). To see a list of all the Amazon SageMaker samples, after creating and opening a notebook instance, choose the **SageMaker Examples** tab. To access the example notebooks that show how to use training metrics, `object2vec_sentence_similarity.ipynb` and `xgboost_abalone.ipynb`., from the **Introduction to Amazon algorithms** section. To open a notebook, choose its **Use** tab, then choose **Create copy**.

Defining Training Metrics

Amazon SageMaker automatically parses the logs for metrics that built-in algorithms emit and sends those metrics to CloudWatch. If you want Amazon SageMaker to parse logs from a custom algorithm and send metrics that the algorithm emits to CloudWatch, you have to specify the metrics that you want Amazon SageMaker to send to CloudWatch when you configure the training job. You specify the name of the metrics that you want to send and the regular expressions that Amazon SageMaker uses to parse the logs that your algorithm emits to find those metrics.

You can specify the metrics that you want to track with the Amazon SageMaker console;, the Amazon SageMaker Python SDK (https://github.com/aws/sagemaker-python-sdk), or the low-level Amazon SageMaker API.

Topics

- Defining Regular Expressions for Metrics (p. 277)
- Defining Training Metrics (Low-level Amazon SageMaker API) (p. 278)
- Defining Training Metrics (Amazon SageMaker Python SDK) (p. 278)
- Define Training Metrics (Console) (p. 279)

Defining Regular Expressions for Metrics

To find a metric, Amazon SageMaker searches the logs that your algorithm emits and finds logs that match the regular expression that you specify for that metric. If you are using your own algorithm, do the following:

- Make sure that the algorithm writes the metrics that you want to capture to logs
- Define a regular expression that accurately searches the logs to capture the values of the metrics that you want to send to CloudWatch metrics.

For example, suppose your algorithm emits metrics for training error and validation error by writing logs similar to the following to `stdout` or `stderr`:

```
Train_error=0.138318;  Valid_error = 0.324557;
```

If you want to monitor both of those metrics in CloudWatch, your `AlgorithmSpecification` would look like the following:

```
"AlgorithmSpecification": {
        "TrainingImage": ContainerName,
        "TrainingInputMode": "File",
        "MetricDefinitions" : [
            {
```

```
            "Name": "train:error",
            "Regex": "Train_error=(.*?);"
        },
             {
            "Name": "validation:error",
            "Regex": "Valid_error=(.*?);"
        }

    ]}
```

In the regex for the `train:error` metric defined above, the first part of the regex finds the exact text "Train_error=", and the expression `(.*?);` captures zero or more of any character until the first semicolon character. In this expression, the parenthesis tell the regex to capture what is inside them, `.` means any character, `*` means zero or more, and `?` means capture only until the first instance of the `;` character.

Defining Training Metrics (Low-level Amazon SageMaker API)

Define the metrics that you want to send to CloudWatch by specifying a list of metric names and regular expressions in the `MetricDefinitions` field of the AlgorithmSpecification (p. 863) input parameter that you pass to the CreateTrainingJob (p. 667) operation. For example, if you want to monitor both the `train:error` and `validation:error` metrics in CloudWatch, your `AlgorithmSpecification` would look like the following:

```
"AlgorithmSpecification": {
        "TrainingImage": ContainerName,
        "TrainingInputMode": "File",
        "MetricDefinitions" : [
            {
            "Name": "train:error",
            "Regex": "Train_error=(.*?);"
        },
             {
            "Name": "validation:error",
            "Regex": "Valid_error=(.*?);"
        }

    ]}
```

For more information about defining and running a training job by using the low-level Amazon SageMaker API, see Create and Run a Training Job (AWS SDK for Python (Boto 3)) (p. 23).

Defining Training Metrics (Amazon SageMaker Python SDK)

Define the metrics that you want to send to CloudWatch by specifying a list of metric names and regular expressions as the `metric_definitions` argument when you initialize an `Estimator` object. For example, if you want to monitor both the `train:error` and `validation:error` metrics in CloudWatch, your `Estimator` initialization would look like the following:

```
estimator =
                Estimator(image_name=ImageName,
                role='SageMakerRole', train_instance_count=1,
                train_instance_type='ml.c4.xlarge',
                train_instance_type='ml.c4.xlarge',
                k=10,
                sagemaker_session=sagemaker_session,
                metric_definitions=[
                   {'Name': 'train:error', 'Regex': 'Train_error=(.*?);'},
                   {'Name': 'validation:error', 'Regex': 'Valid_error=(.*?);'
```

```
                ]
            )
```

For more information about training by using Amazon SageMaker Python SDK estimators, see https://github.com/aws/sagemaker-python-sdk#sagemaker-python-sdk-overview.

Define Training Metrics (Console)

You can define metrics for a custom algorithm in the console when you create a training job by providing the name and regular expression (regex) for **Metrics**.

For example, if you want to monitor both the `train:error` and `validation:error` metrics in CloudWatch, your metric definitions would look like the following:

```
[
            {
            "Name": "train:error",
            "Regex": "Train_error=(.*?);"
        },
             {
            "Name": "validation:error",
            "Regex": "Valid_error=(.*?);"
        }

    ]}
```

Monitoring Training Job Metrics (Console)

You can monitor the metrics that a training job emits in real time in the CloudWatch console.

To monitor training job metrics (CloudWatch console)

1. Open the CloudWatch console at https://console.aws.amazon.com/cloudwatch/.
2. Choose **Metrics**, then choose **/aws/sagemaker/TrainingJobs**.
3. Choose **TrainingJobName**.
4. On the **All metrics** tab, choose the names of the training metrics that you want to monitor.
5. On the **Graphed metrics** tab, configure the graph options. For more information about using CloudWatch graphs, see Graph Metrics in the *Amazon CloudWatch User Guide*.

Monitoring Training Job Metrics (Amazon SageMaker Console)

You can monitor the metrics that a training job emits in real time by using the Amazon SageMaker console.

To monitor training job metrics (Amazon SageMaker console)

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Training jobs**, then choose the training job whose metrics you want to see.
3. Choose **TrainingJobName**.
4. In the **Monitor** section, you can review the graphs of instance utilization and algorithm metrics.

Monitor
Access logs for debugging and progress reporting. View metrics to set alarms, send notifications, or take actions. Learn more
View algorithm metrics
View logs
View instance metrics
2019-01-24 (10:33:57) -
MemoryUtilization
CPUUtilization
DiskUtilization
GPUUtilization
GPUMemoryUtilization
validation:accuracy
train:progress
train:throughput
train:accuracy
validation:cross_entropy
train:cross_entropy

Example: Viewing a Training and Validation Curve

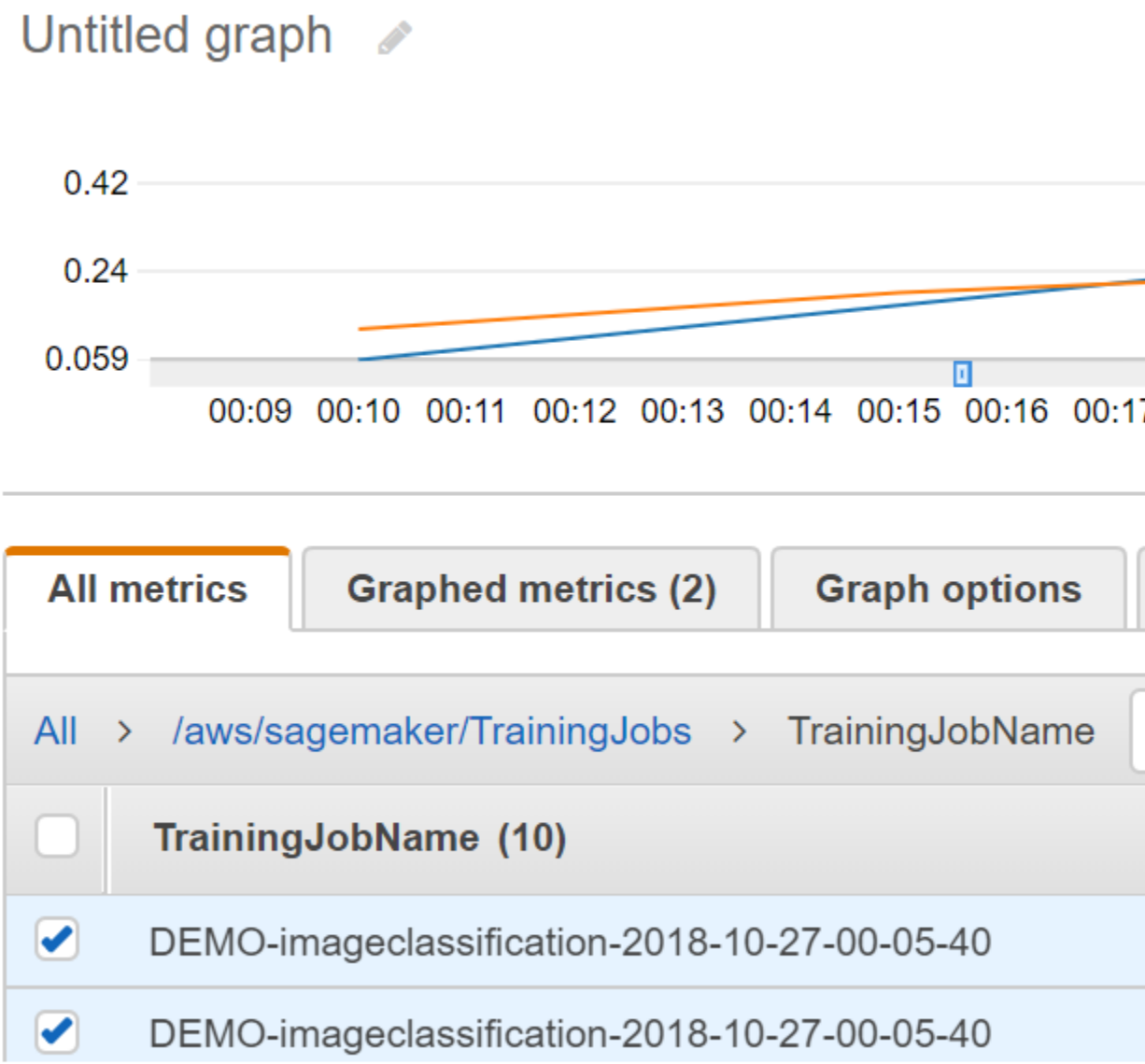
Typically, you split the data that you train your model on into training and validation datasets. You use the training set to train the model parameters that are used to make predictions on the training dataset. Then you test how well the model makes predictions by calculating predictions for the validation set. To analyze the performance of a training job, you commonly plot a training curve against a validation curve.

Viewing a graph that shows the accuracy for both the training and validation sets over time can help you to improve the performance of your model. For example, if training accuracy continues to increase over time, but, at some point, validation accuracy starts to decrease, you are likely overfitting your model. To address this, you can make adjustments to your model, such as increasing regularization.

For this example, you can use the **Image-classification-full-training** example that is in the **Example notebooks** section of your Amazon SageMaker notebook instance. If you don't have an Amazon SageMaker notebook instance, create one by following the instructions at Step 2: Create an Amazon SageMaker Notebook Instance (p. 17). If you prefer, you can follow along with the End-to-End Multiclass Image Classification Example in the example notebook on GitHub. You also need an Amazon S3 bucket to store the training data and for the model output. If you haven't created a bucket to use with Amazon SageMaker, create one by following the instructions at Step 1: Create an Amazon S3 Bucket (p. 17).

To view training and validation error curves

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Notebooks**, and then choose **Notebook instances**.
3. Choose the notebook instance that you want to use, and then choose **Open**.
4. On the dashboard for your notebook instance, choose **SageMaker Examples**.
5. Expand the **Introduction to Amazon Algorithms** section, and then choose **Use** next to **Image-classification-full-training.ipynb**.
6. Choose **Create copy**. Amazon SageMaker creates an editable copy of the **Image-classification-full-training.ipynb** notebook in your notebook instance.
7. In the first code cell of the notebook, replace `<<bucket-name>>` with the name of your S3 bucket.
8. Run all of the cells in the notebook up to the **Deploy** section. You don't need to deploy an endpoint or get inference for this example.
9. After the training job starts, open the CloudWatch console at https://console.aws.amazon.com/cloudwatch/.
10. Choose **Metrics**, then choose **/aws/sagemaker/TrainingJobs**.
11. Choose **TrainingJobName**.
12. On the **All metrics** tab, choose the **train:accuracy** and **validation:accuracy** metrics for the training job that you created in the notebook.
13. On the graph, choose an area that the metric's values to zoom in. You should see something like the following:



Incremental Training in Amazon SageMaker

Over time, you might find that a model generates inference that are not as good as they were in the past. With incremental training, you can use the artifacts from an existing model and use an expanded dataset to train a new model. Incremental training saves both time and resources.

Use incremental training to:

- Train a new model using an expanded dataset that contains an underlying pattern that was not accounted for in the previous training and which resulted in poor model performance.
- Use the model artifacts or a portion of the model artifacts from a popular publicly available model in a training job. You don't need to train a new model from scratch.

- Resume a training job that was stopped.
- Train several variants of a model, either with different hyperparameter settings or using different datasets.

For more information about training jobs, see Train a Model with Amazon SageMaker (p. 4).

You can train incrementally using the Amazon SageMaker console or the Amazon SageMaker Python SDK.

> **Important**
> Only two built-in algorithms currently support incremental training: Object Detection Algorithm (p. 199) and Image Classification Algorithm (p. 108).

Topics

- Perform Incremental Training (Console) (p. 283)
- Perform Incremental Training (API) (p. 285)

Perform Incremental Training (Console)

To complete this procedure, you need:

- The URL of the Amazon Simple Storage Service (Amazon S3) bucket where you've stored the training data.
- The URL of the S3 bucket where you want to store the output of the job.
- The Amazon Elastic Container Registry path where the training code is stored. For more information, see Common Parameters for Built-In Algorithms (p. 58).
- The URL of the S3 bucket where you've stored the model artifacts that you want to use in incremental training. To find the URL for the model artifacts, see the details page of the training job used to create the model. To find the details page, in the Amazon SageMaker console, choose **Inference**, choose **Models**, and then choose the model.

To restart a stopped training job, use the URL to the model artifacts that are stored in the details page as you would with a model or a completed training job.

To perform incremental training (console)

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker.
2. In the navigation pane, choose **Training**, then choose **Training jobs**.
3. Choose **Create training job**.
4. Provide a name for the training job. The name must be unique within an AWS Region in an AWS account. The training job name must have 1 to 63 characters. Valid characters: a-z, A-Z, 0-9, and . : + = @ _ % - (hyphen).
5. Choose the algorithm that you want to use. For information about algorithms, see Use Amazon SageMaker Built-in Algorithms (p. 56).
6. (Optional) For **Resource configuration**, either leave the default values or increase the resource consumption to reduce computation time.

   a. (Optional) For **Instance type**, choose the ML compute instance type that you want to use. In most cases, **ml.m4.xlarge** is sufficient.
   b. For **Instance count**, use the default, 1.
   c. (Optional) For **Additional volume per instance (GB)**, choose the size of the ML storage volume that you want to provision. In most cases, you can use the default, 1. If you are using a large dataset, use a larger size.

7. Provide information about the input data for the training dataset.
   a. For **Channel name**, either leave the default (**`train`**) or enter a more meaningful name for the training dataset, such as **`expanded-training-dataset`**.
   b. For **InputMode**, choose **File**. For incremental training, you need to use file input mode.
   c. For **S3 data distribution type**, choose **FullyReplicated**. This causes each ML compute instance to use a full replicate of the expanded dataset when training incrementally.
   d. If the expanded dataset is uncompressed, set the **Compression type** to **None**. If the expanded dataset is compressed using Gzip, set it to **Gzip**.
   e. (Optional) If you are using File input mode, leave **Content type** empty. For Pipe input mode, specify the appropriate MIME type. *Content type* is the multipurpose internet mail extension (MIME) type of the data.
   f. For **Record wrapper**, if the dataset is saved in RecordIO format, choose **RecordIO**. If your dataset is not saved as a RecordIO formatted file, choose **None**.
   g. For **S3 data type**, if the dataset us stored as a single file, choose **S3Prefix**. If the dataset is stored as several files in a folder, choose **Manifest**.
   h. For **S3 location**, provide the URL to the path where you stored the expanded dataset.
   i. Choose **Done**.
8. To use model artifacts in a training job, you need to add a new channel and provide the needed information about the model artifacts.
   a. For **Input data configuration**, choose **Add channel**.
   b. For **Channel name**, enter **`model`** to identify this channel as the source of the model artifacts.
   c. For **InputMode**, choose **File**. Model artifacts are stored as files.
   d. For **S3 data distribution type**, choose **FullyReplicated**. This indicates that each ML compute instance should use all of the model artifacts for training.
   e. For **Compression type**, choose **None** because we are using a model for the channel.
   f. Leave **Content type** empty. Content type is the multipurpose internet mail extension (MIME) type of the data. For model artifacts, we leave it empty.
   g. Set **Record wrapper** to **None** because model artifacts are not stored in RecordIO format.
   h. For **S3 data type**, if you are using a built-in algorithm or an algorithm that stores the model as a single file, choose **S3Prefix**. If you are using an algorithm that stores the model as several files, choose **Manifest**.
   i. For **S3 location**, provide the URL to the path where you stored the model artifacts. Typically, the model is stored with the name `model.tar.gz`. To find the URL for the model artifacts, in the navigation pane, choose **Inference**, then choose **Models**. From the list of models, choose a model to display its details page. The URL for the model artifacts is listed under **Primary container** .
   j. Choose **Done**.
9. For **Output data configuration**, provide the following information:
   a. For **S3 location**, type the path to the S3 bucket where you want to store the output data.
   b. (Optional) For **Encryption key**, you can add your AWS Key Management Service (AWS KMS) encryption key to encrypt the output data at rest. Provide the key ID or its Amazon Resource Number (ARN). For more information, see KMS-Managed Encryption Keys.
10. (Optional) For **Tags**, add one or more tags to the training job. A *tag* is metadata that you can define and assign to AWS resources. In this case, you can use tags to help you manage your training jobs. A tag consists of a key and a value, which you define. For example, you might want to create a tag with **`Project`** as a key and a value referring to a project that is related to the training job, such as **`Home value forecasts`**.
11. Choose **Create training job**. Amazon SageMaker creates and runs training job.

After the training job has completed, the newly trained model artifacts are stored under the **S3 output path** that you provided in the **Output data configuration** field. To deploy the model to get predictions, see Step 6: Deploy the Model to Amazon SageMaker (p. 26).

Perform Incremental Training (API)

This example shows how to use Amazon SageMaker APIs to train a model using the Amazon SageMaker image classification algorithm and the Caltech 256 Image Dataset, then train a new model using the first one. It uses Amazon S3 for input and output sources. Please see the incremental training sample notebook for more details on using incremental training.

> **Note**
> In this example we used the original datasets in the incremental training, however you can use different datasets, such as ones that contain newly added samples. Upload the new datasets to S3 and make adjustments to the `data_channels` variable used to train the new model.

Get an AWS Identity and Access Management (IAM) role that grants required permissions and initialize environment variables:

```
import sagemaker
from sagemaker import get_execution_role

role = get_execution_role()
print(role)

sess = sagemaker.Session()

bucket=sess.default_bucket()
print(bucket)
prefix = 'ic-incr-training'
```

Get the training image for the image classification algorithm:

```
from sagemaker.amazon.amazon_estimator import get_image_uri

training_image = get_image_uri(sess.boto_region_name, 'image-classification',
 repo_version="latest")
#Display the training image
print (training_image)
```

Download the training and validation datasets, then upload them to Amazon Simple Storage Service (Amazon S3):

```
import os
import urllib.request
import boto3

# Define a download function
def download(url):
    filename = url.split("/")[-1]
    if not os.path.exists(filename):
        urllib.request.urlretrieve(url, filename)

# Download the caltech-256 training and validation datasets
download('http://data.mxnet.io/data/caltech-256/caltech-256-60-train.rec')
download('http://data.mxnet.io/data/caltech-256/caltech-256-60-val.rec')

# Create four channels: train, validation, train_lst, and validation_lst
s3train = 's3://{}/{}/train/'.format(bucket, prefix)
```

```
s3validation = 's3://{}/{}/validation/'.format(bucket, prefix)

# Upload the first files to the train and validation channels
!aws s3 cp caltech-256-60-train.rec $s3train --quiet
!aws s3 cp caltech-256-60-val.rec $s3validation --quiet
```

Define the training hyperparameters:

```
# Define hyperparameters for the estimator
hyperparams = { "num_layers": "18",
                "resize": "32",
                "num_training_samples": "50000",
                "num_classes": "10",
                "image_shape": "3,28,28",
                "mini_batch_size": "128",
                "epochs": "3",
                "learning_rate": "0.1",
                "lr_scheduler_step": "2,3",
                "lr_scheduler_factor": "0.1",
                "augmentation_type": "crop_color",
                "optimizer": "sgd",
                "momentum": "0.9",
                "weight_decay": "0.0001",
                "beta_1": "0.9",
                "beta_2": "0.999",
                "gamma": "0.9",
                "eps": "1e-8",
                "top_k": "5",
                "checkpoint_frequency": "1",
                "use_pretrained_model": "0",
                "model_prefix": "" }
```

Create an estimator object and train the first model using the training and validation datasets:

```
# Fit the base estimator
s3_output_location = 's3://{}/{}/output'.format(bucket, prefix)
ic = sagemaker.estimator.Estimator(training_image,
                                   role,
                                   train_instance_count=1,
                                   train_instance_type='ml.p2.xlarge',
                                   train_volume_size=50,
                                   train_max_run=360000,
                                   input_mode='File',
                                   output_path=s3_output_location,
                                   sagemaker_session=sess,
                                   hyperparameters=hyperparams)

train_data = sagemaker.session.s3_input(s3train, distribution='FullyReplicated',
                                        content_type='application/x-recordio',
 s3_data_type='S3Prefix')
validation_data = sagemaker.session.s3_input(s3validation, distribution='FullyReplicated',
                                             content_type='application/x-recordio',
 s3_data_type='S3Prefix')

data_channels = {'train': train_data, 'validation': validation_data}

ic.fit(inputs=data_channels, logs=True)
```

To use the model to incrementally train another model, create a new estimator object and use the model artifacts (`ic.model_data`, in this example) for the `model_uri` input argument:

```
# Given the base estimator, create a new one for incremental training
```

```
incr_ic = sagemaker.estimator.Estimator(training_image,
                                        role,
                                        train_instance_count=1,
                                        train_instance_type='ml.p2.xlarge',
                                        train_volume_size=50,
                                        train_max_run=360000,
                                        input_mode='File',
                                        output_path=s3_output_location,
                                        sagemaker_session=sess,
                                        hyperparameters=hyperparams,
                                        model_uri=ic.model_data) # This parameter will
 ingest the previous job's model as a new channel
incr_ic.fit(inputs=data_channels, logs=True)
```

After the training job has completed, the newly trained model artifacts are stored under the `S3 output path` that you provided in `Output_path`. To deploy the model to get predictions, see Step 6: Deploy the Model to Amazon SageMaker (p. 26).

Managed Spot Training in Amazon SageMaker

Amazon SageMaker makes it easy to train machine learning models using managed Amazon EC2 Spot instances. Managed spot training can optimize the cost of training models up to 90% over on-demand instances. Amazon SageMaker manages the Spot interruptions on your behalf.

Managed Spot Training uses Amazon EC2 Spot instance to run training jobs instead of on-demand instances. You can specify which training jobs use spot instances and a stopping condition that specifies how long Amazon SageMaker waits for a job to run using Amazon EC2 Spot instances. Metrics and logs generated during training runs are available in CloudWatch.

Spot instances can be interrupted, causing jobs to take longer to start or finish. You can configure your managed spot training job to use checkpoints. Amazon SageMaker copies checkpoint data from a local path to Amazon S3. When the job is restarted, Amazon SageMaker copies the data from Amazon S3 back into the local path. The training can then resume from the last checkpoint instead of restarting. For more information about checkpointing, see Using Checkpoints in Amazon SageMaker (p. 288).

> **Note**
> Unless your training job will complete quickly, we recommend you use checkpointing with managed spot training. SageMaker built-in algorithms and marketplace algorithms that do not checkpoint are currently limited to a `MaxWaitTimeInSeconds` of 3600 seconds (60 minutes).

Topics

- Using Managed Spot Training (p. 287)
- Managed Spot Training Lifecycle (p. 288)

Using Managed Spot Training

To use managed spot training, create a training job. Set `EnableManagedSpotTraining` to `True` and specify the `MaxWaitTimeInSecods`. `MaxWaitTimeInSeconds` must be larger than `MaxRuntimeInSeconds`. For more information about creating a training job, see CreateTrainingJob (p. 667).

You can calculate the savings from using managed spot training using the formula `(1 - BillableTimeInSeconds / TrainingTimeInSeconds) * 100`. For example, if `BillableTimeInSeconds` is 100 and `TrainingTimeInSeconds` is 500, the savings is 80%.

Managed Spot Training Lifecycle

You can monitor a training job using `TrainingJobStatus` and `SecondaryStatus` returned by DescribeTrainingJob (p. 744). The list below shows how `TrainingJobStatus` and `SecondaryStatus` values change depending on the training scenario:

- **Spot instances acquired with no interruption during training**
  1. `InProgress`: `Starting` → `Downloading` → `Training` → `Uploading`
- **Spot instances interrupted once. Later, enough spot instances were acquired to finish the training job.**
  1. `InProgress`: `Starting` → `Downloading` → `Training` → `Interrupted` → `Starting` → `Downloading` → `Training` → `Uploading`
- **Spot instances interrupted twice and `MaxWaitTimeInSeconds` exceeded.**
  1. `InProgress`: `Starting` → `Downloading` → `Training` → `Interrupted` → `Starting` → `Downloading` → `Training` → `Interrupted` → `Downloading` → `Training`
  2. `Stopping`: `Stopping`
  3. `Stopped`: `MaxWaitTimeExceeded`
- **Spot instances were never launched.**
  1. `InProgress`: `Starting`
  2. `Stopping`: `Stopping`
  3. `Stopped`: `MaxWaitTimeExceeded`

Using Checkpoints in Amazon SageMaker

A checkpoint is a snapshot of the state of the model. They can be used with Managed Spot Training. If a training job is interrupted, a snapshot can be used to resume from a previously saved point. This can save training time.

Snapshots are saved to an Amazon S3 location you specify. You can configure the local path to use for snapshots or use the default. When a training job is interrupted, Amazon SageMaker copies the training data to Amazon S3. When the training job is restarted, the checkpoint data is copied to the local path. It can be used to resume at the checkpoint.

To enable checkpoints, provide an Amazon S3 location. You can optionally provide a local path and choose to use a shared folder. The default local path is `/opt/ml/checkpoints/`. For more information, see CreateTrainingJob (p. 667)

Automatic Model Tuning

Amazon SageMaker automatic model tuning, also known as hyperparameter tuning, finds the best version of a model by running many training jobs on your dataset using the algorithm and ranges of hyperparameters that you specify. It then chooses the hyperparameter values that result in a model that performs the best, as measured by a metric that you choose.

For example, suppose that you want to solve a *binary classification* problem on a marketing dataset. Your goal is to maximize the *area under the curve (auc)* metric of the algorithm by training an XGBoost Algorithm (p. 255) model. You don't know which values of the `eta`, `alpha`, `min_child_weight`, and `max_depth` hyperparameters to use to train the best model. To find the best values for these hyperparameters, you can specify ranges of values that Amazon SageMaker hyperparameter tuning

searches to find the combination of values that results in the training job that performs the best as measured by the objective metric that you chose. Hyperparameter tuning launches training jobs that use hyperparameter values in the ranges that you specified, and returns the training job with highest auc.

You can use Amazon SageMaker automatic model tuning with built-in algorithms, custom algorithms, and Amazon SageMaker pre-built containers for machine learning frameworks.

Before you start using hyperparameter tuning, you should have a well-defined machine learning problem, including the following:

- A dataset
- An understanding of the type of algorithm you need to train
- A clear understanding of how you measure success

You should also prepare your dataset and algorithm so that they work in Amazon SageMaker and successfully run a training job at least once. For information about setting up and running a training job, see Get Started (p. 16).

Topics

- How Hyperparameter Tuning Works (p. 289)
- Define Metrics (p. 290)
- Define Hyperparameter Ranges (p. 292)
- Example: Hyperparameter Tuning Job (p. 293)
- Stop Training Jobs Early (p. 302)
- Run a Warm Start Hyperparameter Tuning Job (p. 303)
- Automatic Model Tuning Resource Limits (p. 307)
- Best Practices for Hyperparameter Tuning (p. 308)

How Hyperparameter Tuning Works

Random Search

In a random search, hyperparameter tuning chooses a random combination of values from within the ranges that you specify for hyperparameters for each training job it launches. Because the choice of hyperparameter values doesn't depend on the results of previous training jobs, you can run the maximum number of concurrent training jobs without affecting the performance of the search.

For an example notebook that uses random search, see https://github.com/awslabs/amazon-sagemaker-examples/blob/master/hyperparameter_tuning/xgboost_random_log/hpo_xgboost_random_log.ipynb.

Bayesian Search

Bayesian search treats hyperparameter tuning like a *[regression]* problem. Given a set of input features (the hyperparameters), hyperparameter tuning optimizes a model for the metric that you choose. To solve a regression problem, hyperparameter tuning makes guesses about which hyperparameter combinations are likely to get the best results, and runs training jobs to test these values. After testing the first set of hyperparameter values, hyperparameter tuning uses regression to choose the next set of hyperparameter values to test.

Hyperparameter tuning uses an Amazon SageMaker implementation of Bayesian optimization.

When choosing the best hyperparameters for the next training job, hyperparameter tuning considers everything that it knows about this problem so far. Sometimes it chooses a combination of hyperparameter values close to the combination that resulted in the best previous training job to incrementally improve performance. This allows hyperparameter tuning to exploit the best known results. Other times, it chooses a set of hyperparameter values far removed from those it has tried. This allows it to explore the range of hyperparameter values to try to find new areas that are not well understood. The explore/exploit trade-off is common in many machine learning problems.

For more information about Bayesian optimization, see the following:

Basic Topics on Bayesian Optimization

- A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning
- Practical Bayesian Optimization of Machine Learning Algorithms
- Taking the Human Out of the Loop: A Review of Bayesian Optimization

Speeding up Bayesian Optimization

- Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization
- Google Vizier: A Service for Black-Box Optimization
- Learning Curve Prediction with Bayesian Neural Networks
- Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves

Advanced Modeling and Transfer Learning

- Scalable Hyperparameter Transfer Learning
- Bayesian Optimization with Tree-structured Dependencies
- Bayesian Optimization with Robust Bayesian Neural Networks
- Scalable Bayesian Optimization Using Deep Neural Networks
- Input Warping for Bayesian Optimization of Non-stationary Functions

> **Note**
> Hyperparameter tuning might not improve your model. It is an advanced tool for building machine solutions, and, as such, should be considered part of the scientific development process.
> When you build complex machine learning systems like deep learning neural networks, exploring all of the possible combinations is impractical. Hyperparameter tuning can accelerate your productivity by trying many variations of a model, focusing on the most promising combinations of hyperparameter values within the ranges that you specify. To get good results, you need to choose the right ranges to explore. Because the algorithm itself is stochastic, it's possible that the hyperparameter tuning model will fail to converge on the best answer, even if the best possible combination of values is within the ranges that you choose.

Define Metrics

> **Note**
> When you use one of the Amazon SageMaker built-in algorithms, you don't need to define metrics. Built-in algorithms automatically send metrics to hyperparameter tuning. You do need to choose one of the metrics that the built-in algorithm emits as the objective metric for the tuning job. For a list of metrics that a built-in algorithm emits, see the *Metrics* table for the algorithm in Use Amazon SageMaker Built-in Algorithms (p. 56).

To optimize hyperparameters for a machine learning model, a tuning job evaluates the training jobs it launches by using a metric that the training algorithm writes to logs. Amazon SageMaker hyperparameter tuning parses your algorithm's `stdout` and `stderr` streams to find algorithm metrics, such as loss or validation-accuracy, that show how well the model is performing on the dataset

> **Note**
> These are the same metrics that Amazon SageMaker sends to CloudWatch Logs. For more information, see Log Amazon SageMaker Events with Amazon CloudWatch (p. 466).

If you use your own algorithm for hyperparameter tuning, make sure that your algorithm emits at least one metric by writing evaluation data to `stderr` or `stdout`.

> **Note**
> Hyperparameter tuning sends an additional hyperparameter, `_tuning_objective_metric` to the training algorithm. This hyperparameter specifies the objective metric being used for the hyperparameter tuning job, so that your algorithm can use that information during training.

You can define up to 20 metrics for your tuning job to monitor. You choose one of those metrics to be the objective metric, which hyperparameter tuning uses to evaluate the training jobs. The hyperparameter tuning job returns the training job that returned the best value for the objective metric as the best training job.

You define metrics for a tuning job by specifying a name and a regular expression for each metric that your tuning job monitors. Design the regular expressions to capture the values of metrics that your algorithm emits. You pass these metrics to the CreateHyperParameterTuningJob (p. 638) operation in the `TrainingJobDefinition` parameter as the `MetricDefinitions` field of the `AlgorithmSpecification` field.

The following example defines 4 metrics:

```
=[
    {
        "Name": "loss",
        "Regex": "Loss = (.*?);",
    },
    {
        "Name": "ganloss",
        "Regex": "GAN_loss=(.*?);",
    },
    {
        "Name": "discloss",
        "Regex": "disc_train_loss=(.*?);",
    },
    {
        "Name": "disc-combined",
        "Regex": "disc-combined=(.*?);",
    },
]
```

The following is an example of the log that the algorithm writes:

```
GAN_loss=0.138318;  Scaled_reg=2.654134; disc:[-0.017371,0.102429] real 93.3% gen 0.0%
 disc-combined=0.000000; disc_train_loss=1.374587;  Loss = 16.020744;  Iteration 0 took
 0.704s;  Elapsed=0s
```

Use the regular expression (regex) to match the algorithm's log output and capture the numeric values of metrics. For example, in the regex for the `loss` metric defined above, the first part of the regex finds the exact text "Loss = ", and the expression `(.*?);` captures zero or more of any character until the first semicolon character. In this expression, the parenthesis tell the regex to capture what is inside them, `.` means any character, `*` means zero or more, and `?` means capture only until the first instance of the `;` character.

Choose one of the metrics that you define as the objective metric for the tuning job. If you are using the API, specify the value of the `name` key in the `HyperParameterTuningJobObjective` field of the `HyperParameterTuningJobConfig` parameter that you send to the CreateHyperParameterTuningJob (p. 638) operation.

Define Hyperparameter Ranges

Hyperparameter tuning finds the best hyperparameter values for your model by searching over ranges of hyperparameters. You specify the hyperparameters and range of values over which to search by defining hyperparameter ranges for your tuning job. Choosing hyperparameters and ranges significantly affects the performance of your tuning job. For guidance on choosing hyperparameters and ranges, see Best Practices for Hyperparameter Tuning (p. 308).

To define hyperparameter ranges by using the low-level API, you specify the names of hyperparameters and ranges of values in the `ParameterRanges` field of the `HyperParameterTuningJobConfig` parameter that you pass to the CreateHyperParameterTuningJob (p. 638) operation. The `ParameterRanges` field has three subfields, one for each of the categorical, integer, and continuous hyperparameter ranges. You can define up to 20 hyperparameters to search over. Each value of a categorical hyperparameter range counts as a hyperparameter against the limit. Hyperparameter ranges have the following structure:

```
"ParameterRanges": {
      "CategoricalParameterRanges": [
        {
          "Name": "tree_method",
          "Values": ["auto", "exact", "approx", "hist"]
        }
      ],
      "ContinuousParameterRanges": [
        {
          "Name": "eta",
          "MaxValue" : "0.5",
          "MinValue": "0",
          "ScalingType": "Auto"
        }
      ],
      "IntegerParameterRanges": [
        {
          "Name": "max_depth",
          "MaxValue": "10",
          "MinValue": "1",
          "ScalingType": "Auto"
        }
      ]
    }
```

Hyperparameter Scaling

For integer and continuous hyperparameter ranges, you can choose the scale you want hyperparameter tuning to use to search the range of values by specifying a value for the `ScalingType` field of the hyperparameter range. You can choose from the following scaling types:

Auto

Amazon SageMaker hyperparameter tuning chooses the best scale for the hyperparameter.

Linear

Hyperparameter tuning searches the values in the hyperparameter range by using a linear scale. Typically, you choose this if the range of all values from the lowest to the highest is relatively small

(within one order of magnitude), because uniformly searching values from the range will give you a reasonable exploration of the entire range.

Logarithmic

Hyperparemeter tuning searches the values in the hyperparameter range by using a logarithmic scale.

Logarithmic scaling works only for ranges that have only values greater than 0.

Choose logarithmic scaling when you are searching a range that spans several orders of magnitude. For example, if you are tuning a Tune a Linear Learner Model (p. 162) model, and you specify a range of values between .0001 and 1.0 for the `learning_rate` hyperparameter, searching uniformly on a logarithmic scale gives you a better sample of the entire range than searching on a linear scale would, because searching on a linear scale would, on average, devote 90 percent of your training budget to only the values between .1 and 1.0, leaving only 10 percent of your training budget for the values between .0001 and .1.

ReverseLogarithmic

Hyperparemeter tuning searches the values in the hyperparameter range by using a reverse logarithmic scale. reverse logarithmic scaling is supported only for continuous hyperparameter ranges. It is not supported for integer hyperparameter ranges.

Reverse logarithmic scaling works only for ranges that are entirely within the range 0<=x<1.0.

Choose reverse logarithmic scaling when you are searching a range that is highly sensitive to small changes that are very close to 1.

For an example notebook that uses hyperparameter scaling, see https://github.com/awslabs/amazon-sagemaker-examples/blob/master/hyperparameter_tuning/xgboost_random_log/hpo_xgboost_random_log.ipynb.

Example: Hyperparameter Tuning Job

This example shows how to create a new notebook for configuring and launching a hyperparameter tuning job. The tuning job uses the XGBoost Algorithm (p. 255) to train a model to predict whether a customer will enroll for a term deposit at a bank after being contacted by phone.

You use the low-level AWS SDK for Python (Boto) to configure and launch the hyperparameter tuning job, and the AWS Management Console to monitor the status of hyperparameter training jobs. You can also use the Amazon SageMaker high-level Amazon SageMaker Python SDK to configure, run, monitor, and analyze hyperparameter tuning jobs. For more information, see https://github.com/aws/sagemaker-python-sdk.

Prerequisites

To run the code in this example, you need

- An AWS account and an administrator user (p. 14)
- An Amazon S3 bucket for storing your training dataset and the model artifacts created during training (p. 17)
- A running Amazon SageMaker notebook instance (p. 17)

Topics

- Create a Notebook (p. 294)

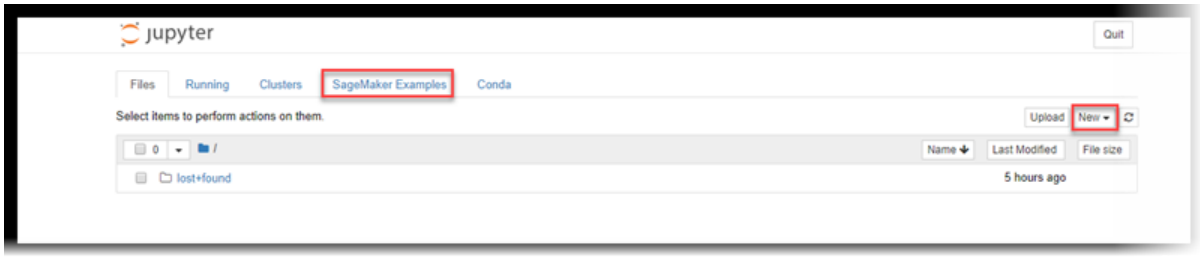
- Get the Amazon Sagemaker Boto 3 Client (p. 294)
- Get the Amazon SageMaker Execution Role (p. 295)
- Specify a Bucket and Data Output Location (p. 295)
- Download, Prepare, and Upload Training Data (p. 295)
- Configure and Launch a Hyperparameter Tuning Job (p. 296)
- Monitor the Progress of a Hyperparameter Tuning Job (p. 299)
- Clean up (p. 301)

Create a Notebook

Create a Jupyter notebook that contains a preinstalled environment with the default Anaconda installation and Python3.

To create a Jupyter notebook

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/
   .
2. Open a running notebook instance, by choosing **Open** next to its name. The Jupyter notebook server page appears:



3. To create a notebook, choose **Files**, **New**, and **conda_python3**. .
4. Name the notebook.

Next Step

Get the Amazon Sagemaker Boto 3 Client (p. 294)

Get the Amazon Sagemaker Boto 3 Client

Import libraries and get a Boto3 client, which you use to call the hyperparameter tuning APIs.

In the new Jupyter notebook, type the following code:

```
import sagemaker
import boto3
from sagemaker.predictor import csv_serializer    # Converts strings for HTTP POST requests
 on inference

import numpy as np                                # For performing matrix operations and
 numerical processing
import pandas as pd                               # For manipulating tabular data
from time import gmtime, strftime
import os
```

```
region = boto3.Session().region_name
smclient = boto3.Session().client('sagemaker')
```

Next Step

Get the Amazon SageMaker Execution Role (p. 295)

Get the Amazon SageMaker Execution Role

Get the execution role for the notebook instance. This is the IAM role that you created when you created your notebook instance. You pass the role to the tuning job.

```
from sagemaker import get_execution_role

role = get_execution_role()
print(role)
```

Next Step

Specify a Bucket and Data Output Location (p. 295)

Specify a Bucket and Data Output Location

Specify the name of the Amazon S3 bucket where you want to store the output of the training jobs that the tuning job launches. The name of the bucket must contain **`sagemaker`**, and be globally unique. The bucket must be in the same AWS Region as the notebook instance that you use for this example. You can use the bucket that you created when you set up Amazon SageMaker, or you can create a new bucket. For information, see Step 1: Create an Amazon S3 Bucket (p. 17).

> **Note**
> The name of the bucket doesn't need to contain **`sagemaker`** if the role that you use to run the hyperparameter tuning job has a policy that gives the SageMaker service principle `S3FullAccess` permission.

`prefix` is the path within the bucket where Amazon SageMaker stores the output from training jobs.

```
bucket = 'sagemaker-MyBucket'                               # Replace with the name of your
 S3 bucket
prefix = 'sagemaker/DEMO-automatic-model-tuning-xgboost-dm'
```

Next Step

Download, Prepare, and Upload Training Data (p. 295)

Download, Prepare, and Upload Training Data

For this example, you use a training dataset of information about bank customers that includes the customer's job, marital status, and how they were contacted during the bank's direct marketing campaign. To use a dataset for a hyperparameter tuning job, you download it, transform the data, and then upload it to an Amazon S3 bucket.

For more information about the dataset and the data transformation that the example performs, see the *hpo_xgboost_direct_marketing_sagemaker_APIs* notebook in the **Hyperparameter Tuning** section of the **SageMaker Examples** tab in your notebook instance.

Download and Explore the Training Dataset

To download and explore the dataset, run the following code in your notebook:

```
!wget -N https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank-
additional.zip
!unzip -o bank-additional.zip
data = pd.read_csv('./bank-additional/bank-additional-full.csv', sep=';')
pd.set_option('display.max_columns', 500)     # Make sure we can see all of the columns
pd.set_option('display.max_rows', 5)         # Keep the output on one page
data
```

Prepare and Upload Data

Before creating the hyperparameter tuning job, prepare the data and upload it to an S3 bucket where the hyperparameter tuning job can access it.

Run the following code in your notebook:

```
data['no_previous_contact'] = np.where(data['pdays'] == 999, 1, 0)
        # Indicator variable to capture when pdays takes a value of 999
data['not_working'] = np.where(np.in1d(data['job'], ['student', 'retired', 'unemployed']),
 1, 0)   # Indicator for individuals not actively employed
model_data = pd.get_dummies(data)
        # Convert categorical variables to sets of indicators
model_data
model_data = model_data.drop(['duration', 'emp.var.rate', 'cons.price.idx',
 'cons.conf.idx', 'euribor3m', 'nr.employed'], axis=1)

train_data, validation_data, test_data = np.split(model_data.sample(frac=1,
 random_state=1729), [int(0.7 * len(model_data)), int(0.9*len(model_data))])

pd.concat([train_data['y_yes'], train_data.drop(['y_no', 'y_yes'], axis=1)],
 axis=1).to_csv('train.csv', index=False, header=False)
pd.concat([validation_data['y_yes'], validation_data.drop(['y_no', 'y_yes'], axis=1)],
 axis=1).to_csv('validation.csv', index=False, header=False)
pd.concat([test_data['y_yes'], test_data.drop(['y_no', 'y_yes'], axis=1)],
 axis=1).to_csv('test.csv', index=False, header=False)

boto3.Session().resource('s3').Bucket(bucket).Object(os.path.join(prefix, 'train/
train.csv')).upload_file('train.csv')
boto3.Session().resource('s3').Bucket(bucket).Object(os.path.join(prefix, 'validation/
validation.csv')).upload_file('validation.csv')
```

Next Step

Configure and Launch a Hyperparameter Tuning Job (p. 296)

Configure and Launch a Hyperparameter Tuning Job

To configure and launch a hyperparameter tuning job, complete the following steps.

Topics

- Specify the Hyperparameter Tuning Job Settings (p. 297)
- Configure the Training Jobs (p. 298)
- Name and Launch the Hyperparameter Tuning Job (p. 299)
- Next Step (p. 299)

Specify the Hyperparameter Tuning Job Settings

To specify settings for the hyperparameter tuning job, you define a JSON object. You pass the object as the value of the `HyperParameterTuningJobConfig` parameter to CreateHyperParameterTuningJob (p. 638) when you create the tuning job.

In this JSON object, you specify:

- The ranges of hyperparameters that you want to tune. For more information, see Define Hyperparameter Ranges (p. 292)
- The limits of the resource that the hyperparameter tuning job can consume.
- The objective metric for the hyperparameter tuning job. An *objective metric* is the metric that the hyperparameter tuning job uses to evaluate the training job that it launches.

  Note
  To use your own algorithm for hyperparameter tuning, you need to define metrics for your algorithm. For information,see Define Metrics (p. 290).

The hyperparameter tuning job defines ranges for the `eta`, `alpha`, `min_child_weight`, and `max_depth` hyperparameters of the XGBoost Algorithm (p. 255) built-in algorithm. The objective metric for the hyperparameter tuning job maximizes the `validation:auc` metric that the algorithm sends to CloudWatch Logs.

```
tuning_job_config = {
    "ParameterRanges": {
      "CategoricalParameterRanges": [],
      "ContinuousParameterRanges": [
        {
          "MaxValue": "1",
          "MinValue": "0",
          "Name": "eta"
        },
        {
          "MaxValue": "2",
          "MinValue": "0",
          "Name": "alpha"
        },
        {
          "MaxValue": "10",
          "MinValue": "1",
          "Name": "min_child_weight"
        }
      ],
      "IntegerParameterRanges": [
        {
          "MaxValue": "10",
          "MinValue": "1",
          "Name": "max_depth"
        }
      ]
    },
    "ResourceLimits": {
      "MaxNumberOfTrainingJobs": 20,
      "MaxParallelTrainingJobs": 3
    },
    "Strategy": "Bayesian",
    "HyperParameterTuningJobObjective": {
      "MetricName": "validation:auc",
      "Type": "Maximize"
    }
  }
```

Configure the Training Jobs

To configure the training jobs that the tuning job launches, define a JSON object that you pass as the value of the `TrainingJobDefinition` parameter of the CreateHyperParameterTuningJob (p. 638) call.

In this JSON object, you specify:

- Optional—Metrics that the training jobs emit.

  Note
  Define metrics only when you use a custom training algorithm. Because this example uses a built-in algorithm, you don't specify metrics. For information about defining metrics, see Define Metrics (p. 290).
- The container image that specifies the training algorithm.
- The input configuration for your training and test data.
- The storage location for the algorithm's output. Specify the S3 bucket where you want to store the output of the training jobs.
- The values of algorithm hyperparameters that are not tuned in the tuning job.
- The type of instance to use for the training jobs.
- The stopping condition for the training jobs. This is the maximum duration for each training job.

In this example, we set static values for the `eval_metric`, `num_round`, `objective`, `rate_drop`, and `tweedie_variance_power` parameters of the XGBoost Algorithm (p. 255) built-in algorithm.

```
from sagemaker.amazon.amazon_estimator import get_image_uri
training_image = get_image_uri(boto3.Session().region_name, 'xgboost')

s3_input_train = 's3://{}/{}/train'.format(bucket, prefix)
s3_input_validation ='s3://{}/{}/validation/'.format(bucket, prefix)

training_job_definition = {
    "AlgorithmSpecification": {
      "TrainingImage": training_image,
      "TrainingInputMode": "File"
    },
    "InputDataConfig": [
      {
        "ChannelName": "train",
        "CompressionType": "None",
        "ContentType": "csv",
        "DataSource": {
          "S3DataSource": {
            "S3DataDistributionType": "FullyReplicated",
            "S3DataType": "S3Prefix",
            "S3Uri": s3_input_train
          }
        }
      },
      {
        "ChannelName": "validation",
        "CompressionType": "None",
        "ContentType": "csv",
        "DataSource": {
          "S3DataSource": {
            "S3DataDistributionType": "FullyReplicated",
            "S3DataType": "S3Prefix",
            "S3Uri": s3_input_validation
          }
```

```
        }
      }
    ],
    "OutputDataConfig": {
      "S3OutputPath": "s3://{}/{}/output".format(bucket,prefix)
    },
    "ResourceConfig": {
      "InstanceCount": 2,
      "InstanceType": "ml.c4.2xlarge",
      "VolumeSizeInGB": 10
    },
    "RoleArn": role,
    "StaticHyperParameters": {
      "eval_metric": "auc",
      "num_round": "100",
      "objective": "binary:logistic",
      "rate_drop": "0.3",
      "tweedie_variance_power": "1.4"
    },
    "StoppingCondition": {
      "MaxRuntimeInSeconds": 43200
    }
}
```

Name and Launch the Hyperparameter Tuning Job

Now you can provide a name for the hyperparameter tuning job and then launch it by calling the CreateHyperParameterTuningJob (p. 638) API. Pass `tuning_job_config`, and `training_job_definition` that you created in previous steps as the values of the parameters.

```
tuning_job_name = "MyTuningJob"
smclient.create_hyper_parameter_tuning_job(HyperParameterTuningJobName = tuning_job_name,
                                           HyperParameterTuningJobConfig =
 tuning_job_config,
                                           TrainingJobDefinition = training_job_definition)
```

Next Step

Monitor the Progress of a Hyperparameter Tuning Job (p. 299)

Monitor the Progress of a Hyperparameter Tuning Job

To monitor the progress of a hyperparameter tuning job and the training jobs that it launches, use the Amazon SageMaker console.

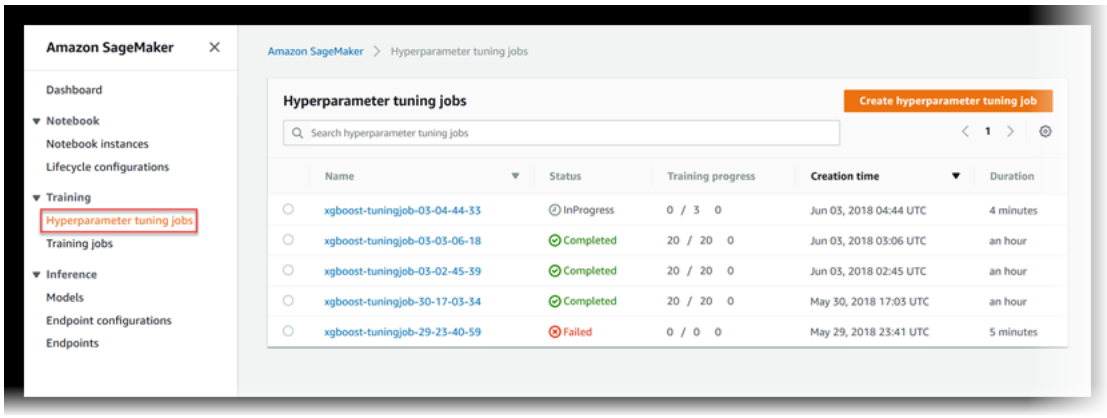
Topics

- View the Status of the Hyperparameter Tuning Job (p. 299)
- View the Status of the Training Jobs (p. 300)
- View the Best Training Job (p. 301)

View the Status of the Hyperparameter Tuning Job

To view the status of the hyperparameter tuning job

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/ .
2. Choose **Hyperparameter tuning jobs**.



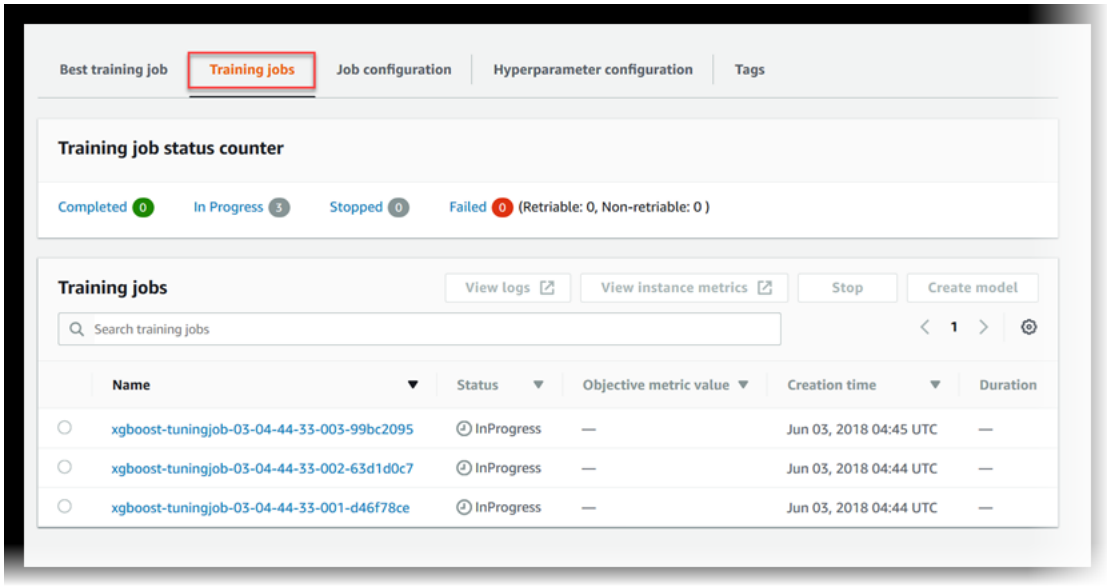
3. In the list of hyperparameter tuning jobs, check the status of the hyperparameter tuning job you launched. A tuning job can be:

   - `Completed`—The hyperparameter tuning job successfully completed.
   - `InProgress`—The hyperparameter tuning job is in progress. One or more training jobs are still running.
   - `Failed`—The hyperparameter tuning job failed.
   - `Stopped`—The hyperparameter tuning job was manually stopped before it completed. All training jobs that the hyperparameter tuning job launched are stopped.
   - `Stopping`—The hyperparameter tuning job is in the process of stopping.

View the Status of the Training Jobs

To view the status of the training jobs that the hyperparameter tuning job launched

1. In the list of hyperparameter tuning jobs, choose the job that you launched.
2. Choose **Training jobs**.



3. View the status of each training job. To see more details about a job, choose it in the list of training jobs. To view a summary of the status of all of the training jobs that the hyperparameter tuning job launched, see **Training job status counter**.

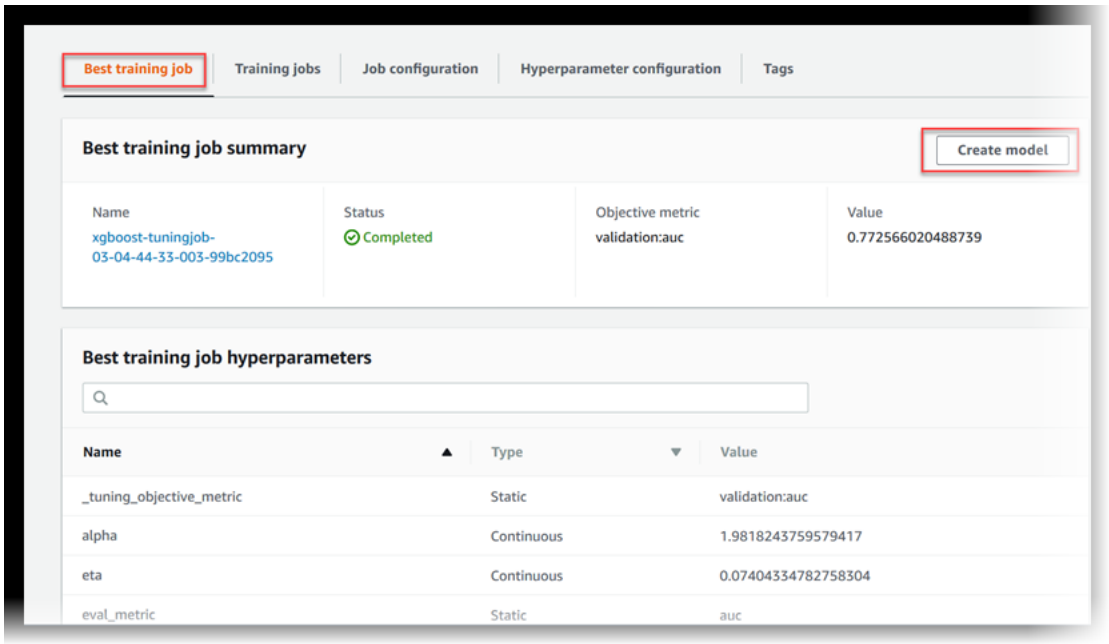
A training job can be:

- `Completed`—The training job successfully completed.
- `InProgress`—The training job is in progress.
- `Stopped`—The training job was manually stopped before it completed.
- `Failed (Retriable)`—The training job failed, but can be retried. A failed training job can be retried only if it failed because an internal service error occurred.
- `Failed (Non-retriable)`—The training job failed and can't be retried. A failed training job can't be retried when a client error occurs.

View the Best Training Job

A hyperparameter tuning job uses the objective metric that each training job returns to evaluate training jobs. While the hyperparameter tuning job is in progress, the best training job is the one that has returned the best objective metric so far. After the hyperparameter tuning job is complete, the best training job is the one that returned the best objective metric.

To view the best training job, choose **Best training job**.



To deploy the best training job as a model that you can host at an Amazon SageMaker endpoint, choose **Create model**.

Next Step

Clean up (p. 301)

Clean up

To avoid incurring unnecessary charges, when you are done with the example, use the AWS Management Console to delete the resources that you created for it.

> **Note**
> If you plan to explore other examples, you might want to keep some of these resources, such as your notebook instance, S3 bucket, and IAM role.

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/ and delete the notebook instance. Stop the instance before deleting it.
2. Open the Amazon S3 console at https://console.aws.amazon.com/s3/ and delete the bucket that you created to store model artifacts and the training dataset.
3. Open the IAM console at https://console.aws.amazon.com/iam/ and delete the IAM role. If you created permission policies, you can delete them, too.
4. Open the Amazon CloudWatch console at https://console.aws.amazon.com/cloudwatch/ and delete all of the log groups that have names starting with `/aws/sagemaker/`.

Stop Training Jobs Early

Stop the training jobs that a hyperparameter tuning job launches early when they are not improving significantly as measured by the objective metric. Stopping training jobs early can help reduce compute time and helps you avoid overfitting your model. To configure a hyperparameter tuning job to stop training jobs early, do one of the following:

- If you are using the AWS SDK for Python (Boto 3), set the `TrainingJobEarlyStoppingType` field of the HyperParameterTuningJobConfig (p. 922) object that you use to configure the tuning job to `AUTO`.
- If you are using the Amazon SageMaker Python SDK, set the `early_stopping_type` parameter of the HyperParameterTuner object to `Auto`.
- In the Amazon SageMaker console, in the **Create hyperparameter tuning job** workflow, under **Early stopping**, choose **Auto**.

For a sample notebook that demonstrates how to use early stopping, see https://github.com/awslabs/amazon-sagemaker-examples/blob/master/hyperparameter_tuning/image_classification_early_stopping/hpo_image_classification_early_stopping.ipynb or open the `hpo_image_classification_early_stopping.ipynb` notebook in the **Hyperparameter Tuning** section of the **SageMaker Examples** in a notebook instance. For information about using sample notebooks in a notebook instance, see Use Example Notebooks (p. 42).

How Early Stopping Works

When you enable early stopping for a hyperparameter tuning job, Amazon SageMaker evaluates each training job the hyperparameter tuning job launches as follows:

- After each epoch of training, get the value of the objective metric.
- Compute the running average of the objective metric for all previous training jobs up to the same epoch, and then compute the median of all of the running averages.
- If the value of the objective metric for the current training job is worse (higher when minimizing or lower when maximizing the objective metric) than the median value of running averages of the objective metric for previous training jobs up to the same epoch, Amazon SageMaker stops the current training job.

Algorithms That Support Early Stopping

To support early stopping, an algorithm must emit objective metrics for each epoch. The following built-in Amazon SageMaker algorithms support early stopping:

- Linear Learner Algorithm (p. 162)—Supported only if you use `objective_loss` as the objective metric.
- XGBoost Algorithm (p. 255)

- Image Classification Algorithm (p. 108)
- Object Detection Algorithm (p. 199)
- Sequence-to-Sequence Algorithm (p. 242)
- IP Insights Algorithm (p. 131)

> **Note**
> This list of built-in algorithms that support early stopping is current as of December 13, 2018. Other built-in algorithms might support early stopping in the future. If an algorithm emits a metric that can be used as an objective metric for a hyperparameter tuning job (preferably a validation metric), then it supports early stopping.

To use early stopping with your own algorithm, you must write your algorithms such that it emits the value of the objective metric after each epoch. The following list shows how you can do that in different frameworks:

TensorFlow

Use the `tf.contrib.learn.monitors.ValidationMonitor` class. For information, see https://www.tensorflow.org/api_docs/python/tf/contrib/learn/monitors.

MXNet

Use the `mxnet.callback.LogValidationMetricsCallback`. For information, see https://mxnet.apache.org/api/python/callback/callback.html.

Chainer

Extend chainer by using the `extensions.Evaluator` class. For information, see https://docs.chainer.org/en/v1.24.0/reference/extensions.html#evaluator.

PyTorch and Spark

There is no high-level support. You must explicitly write your training code so that it computes objective metrics and writes them to logs after each epoch.

Run a Warm Start Hyperparameter Tuning Job

Use warm start to start a hyperparameter tuning job using one or more previous tuning jobs as a starting point. The results of previous tuning jobs are used to inform which combinations of hyperparameters to search over in the new tuning job. Hyperparameter tuning uses either Bayesian or random search to choose combinations of hyperparameter values from ranges that you specify. For more information, see How Hyperparameter Tuning Works (p. 289). Using information from previous hyperparameter tuning jobs can help increase the performance of the new hyperparameter tuning job by making the search for the best combination of hyperparameters more efficient.

> **Note**
> Warm start tuning jobs typically take longer to start than standard hyperparameter tuning jobs, because the results from the parent jobs have to be loaded before the job can start. The increased time depends on the total number of training jobs launched by the parent jobs.

Reasons you might want to consider warm start include:

- You want to gradually increase the number of training jobs over several tuning jobs based on the results you see after each iteration.
- You get new data, and want to tune a model using the new data.
- You want to change the ranges of hyperparameters that you used in a previous tuning job, change static hyperparameters to tunable, or change tunable hyperparameters to static values.
- You stopped a previous hyperparameter job early or it stopped unexpectedly.

Topics

- Types of Warm Start Tuning Jobs (p. 304)
- Warm Start Tuning Restrictions (p. 304)
- Warm Start Tuning Sample Notebook (p. 305)
- Create a Warm Start Tuning Job (p. 305)

Types of Warm Start Tuning Jobs

There are two different types of warm start tuning jobs:

`IDENTICAL_DATA_AND_ALGORITHM`

The new hyperparameter tuning job uses the same input data and training image as the parent tuning jobs. You can change the hyperparameter ranges to search and the maximum number of training jobs that the hyperparameter tuning job launches. You can also change hyperparameters from tunable to static, and from static to tunable, but the total number of static plus tunable hyperparameters must remain the same as it is in all parent jobs. You cannot use a new version of the training algorithm, unless the changes in the new version do not affect the algorithm itself. For example, changes that improve logging or adding support for a different data format are allowed.

Use identical data and algorithm when you use the same training data as you used in a previous hyperparameter tuning job, but you want to increase the total number of training jobs or change ranges or values of hyperparameters.

When you run an warm start tuning job of type `IDENTICAL_DATA_AND_ALGORITHM`, there is an additional field in the response to DescribeHyperParameterTuningJob (p. 715) named `OverallBestTrainingJob`. The value of this field is the TrainingJobSummary (p. 1019) for the training job with the best objective metric value of all training jobs launched by this tuning job and all parent jobs specified for the warm start tuning job.

`TRANSFER_LEARNING`

The new hyperparameter tuning job can include input data, hyperparameter ranges, maximum number of concurrent training jobs, and maximum number of training jobs that are different than those of its parent hyperparameter tuning jobs. You can also change hyperparameters from tunable to static, and from static to tunable, but the total number of static plus tunable hyperparameters must remain the same as it is in all parent jobs. The training algorithm image can also be a different version from the version used in the parent hyperparameter tuning job. When you use transfer learning, changes in the dataset or the algorithm that significantly affect the value of the objective metric might reduce the usefulness of using warm start tuning.

Warm Start Tuning Restrictions

The following restrictions apply to all warm start tuning jobs:

- A tuning job can have a maximum of 5 parent jobs, and all parent jobs must be in a terminal state (`Completed`, `Stopped`, or `Failed`) before you start the new tuning job.
- The objective metric used in the new tuning job must be the same as the objective metric used in the parent jobs.
- The total number of static plus tunable hyperparameters must remain the same between parent jobs and the new tuning job. Because of this, if you think you might want to use a hyperparameter as tunable in a future warm start tuning job, you should add it as a static hyperparameter when you create a tuning job.
- The type of each hyperparameter (continuous, integer, categorical) must not change between parent jobs and the new tuning job.

- The number of total changes from tunable hyperparameters in the parent jobs to static hyperparameters in the new tuning job, plus the number of changes in the values of static hyperparameters cannot be more than 10. Each value in a categorical hyperparameter counts against this limit. For example, if the parent job has a tunable categorical hyperparameter with the possible values `red` and `blue`, you change that hyperparameter to static in the new tuning job, that counts as 2 changes against the allowed total of 10. If the same hyperparameter had a static value of `red` in the parent job, and you change the static value to `blue` in the new tuning job, it also counts as 2 changes.
- Warm start tuning is not recursive. For example, if you create `MyTuningJob3` as a warm start tuning job with `MyTuningJob2` as a parent job, and `MyTuningJob2` is itself an warm start tuning job with a parent job `MyTuningJob1`, the information that was learned when running `MyTuningJob1` is not used for `MyTuningJob3`. If you want to use the information from `MyTuningJob1`, you must explicitly add it as a parent for `MyTuningJob3`.
- The training jobs launched by every parent job in a warm start tuning job count against the 500 maximum training jobs for a tuning job.
- Hyperparameter tuning jobs created before October 1, 2018 cannot be used as parent jobs for warm start tuning jobs.

Warm Start Tuning Sample Notebook

For a sample notebook that shows how to use warm start tuning, see https://github.com/awslabs/amazon-sagemaker-examples/blob/master/hyperparameter_tuning/image_classification_warmstart/hpo_image_classification_warmstart.ipynb. For instructions how to create and access Jupyter notebook instances that you can use to run the example in Amazon SageMaker, see Use Example Notebooks (p. 42). Once you have created a notebook instance and opened it, select the **SageMaker Examples** tab to see a list of all the Amazon SageMaker samples. The warm start tuning example notebook is located in the **Hyperparameter tuning** section, and is named `hpo_image_classification_warmstart.ipynb`. To open a notebook, click on its **Use** tab and select **Create copy**.

Create a Warm Start Tuning Job

You can use either the low-level AWS SDK for Python (Boto 3) or the high-level Amazon SageMaker Python SDK to create a warm start tuning job.

Topics

- Create a Warm Start Tuning Job (Low-level Amazon SageMaker API for Python (Boto 3)) (p. 305)
- Create a Warm Start Tuning Job (Amazon SageMaker Python SDK) (p. 306)

Create a Warm Start Tuning Job (Low-level Amazon SageMaker API for Python (Boto 3))

To use warm start tuning, you specify the values of a HyperParameterTuningJobWarmStartConfig (p. 927) object, and pass that as the `WarmStartConfig` field in a call to CreateHyperParameterTuningJob (p. 638).

The following code shows how to create a HyperParameterTuningJobWarmStartConfig (p. 927) object and pass it to CreateHyperParameterTuningJob (p. 638) job by using the low-level Amazon SageMaker API for Python (Boto 3).

Create the `HyperParameterTuningJobWarmStartConfig` object:

```
warm_start_config = {
          "ParentHyperParameterTuningJobs" : [
          {"HyperParameterTuningJobName" : 'MyParentTuningJob'}
```

```
          ],
          "WarmStartType" : "IdenticalDataAndAlgorithm"
}
```

Create the warm start tuning job:

```
smclient = boto3.Session().client('sagemaker')
smclient.create_hyper_parameter_tuning_job(HyperParameterTuningJobName =
 'MyWarmStartTuningJob',
   HyperParameterTuningJobConfig = tuning_job_config, # See notebook for tuning
 configuration
   TrainingJobDefinition = training_job_definition, # See notebook for job definition
   WarmStartConfig = warm_start_config)
```

Create a Warm Start Tuning Job (Amazon SageMaker Python SDK)

To use the Amazon SageMaker Python SDK to run a warm start tuning job, you:

- Specify the parent jobs and the warm start type by using a `WarmStartConfig` object.
- Pass the `WarmStartConfig` object as the value of the `warm_start_config` argument of a HyperparameterTuner object.
- Call the `fit` method of the `HyperparameterTuner` object.

For more information about using the Amazon SageMaker Python SDK for hyperparameter tuning, see https://github.com/aws/sagemaker-python-sdk#sagemaker-automatic-model-tuning.

This example uses an estimator that uses the Image Classification Algorithm (p. 108) algorithm for training. The following code sets the hyperparameter ranges that the warm start tuning job searches within to find the best combination of values. For information about setting hyperparameter ranges, see Define Hyperparameter Ranges (p. 292).

```
hyperparameter_ranges = {'learning_rate': ContinuousParameter(0.0, 0.1),
                         'momentum': ContinuousParameter(0.0, 0.99)}
```

The following code configures the warm start tuning job by creating a `WarmStartConfig` object.

```
from sagemaker.tuner import WarmStartConfig,
          WarmStartTypes

parent_tuning_job_name = "MyParentTuningJob"
warm_start_config = WarmStartConfig(type=WarmStartTypes.IDENTICAL_DATA_AND_ALGORITHM,
 parents={parent_tuning_job_name})
```

Now set the values for static hyperparameters, which are hyperparameters that keep the same value for every training job that the warm start tuning job launches. In the following code, `imageclassification` is an estimator that was created previously.

```
imageclassification.set_hyperparameters(num_layers=18,
                                        image_shape='3,224,224',
                                        num_classes=257,
                                        num_training_samples=15420,
                                        mini_batch_size=128,
                                        epochs=30,
                                        optimizer='sgd',
                                        top_k='2',
                                        precision_dtype='float32',
                                        augmentation_type='crop')
```

Now create the `HyperparameterTuner` object and pass the `WarmStartConfig` object that you previously created as the `warm_start_config` argument.

```
tuner_warm_start = HyperparameterTuner(imageclassification,
                            'validation:accuracy',
                            hyperparameter_ranges,
                            objective_type='Maximize',
                            max_jobs=10,
                            max_parallel_jobs=2,
                            base_tuning_job_name='warmstart',
                            warm_start_config=warm_start_config)
```

Finally, call the `fit` method of the `HyperparameterTuner` object to launch the warm start tuning job.

```
tuner_warm_start.fit(
        {'train': s3_input_train, 'validation': s3_input_validation},
        include_cls_metadata=False)
```

Automatic Model Tuning Resource Limits

Amazon SageMaker sets default limits for the following resources:

- Number of concurrent hyperparameter tuning jobs - 100
- Number of hyperparameters that can be searched - 20

  Note
  Every possible value in a categorical hyperparameter counts against this limit.
- Number of metrics defined per hyperparameter tuning job - 20
- Number of concurrent training jobs per hyperparameter tuning job - 10
- Number of training jobs per hyperparameter tuning job - 500
- Maximum run time for a hyperparameter tuning job - 30 days

When you plan hyperparameter tuning jobs, you also have to take the limits on training resources into account. For information about the default resource limits for Amazon SageMaker training jobs, see Amazon SageMaker Limits. Every concurrent training instance that all of your hyperparameter tuning jobs run on count against the total number of training instances allowed. For example, suppose you run 10 concurrent hyperparameter tuning jobs. Each of those hyperparameter tuning jobs runs 100 total training jobs, and runs 20 concurrent training jobs. Each of those traning jobs runs on one **ml.m4.xlarge** instance. The following limits apply:

- Number of concurrent hyperparameter tuning jobs - You don't need to increase the limit, because 10 tuning jobs is below the limit of 100.
- Number of training jobs per hyperparameter tuning job - You don't need to increase the limit, because 100 training jobs is below the limit of 500.
- Number of concurrent training jobs per hyperparameter tuning job - You need to request a limit increase to 20, because the default limit is 10.
- Amazon SageMaker training **ml.m4.xlarge** instances - You need to request limit increase to 200, because you have 10 hyperparameter tuning jobs, with each of them running 20 concurrent training jobs. The default limit is 20 instances.
- Amazon SageMaker training total instance count - You need to request a limit increase to 200, because you have 10 hyperparameter tuning jobs, with each of them running 20 concurrent training jobs. The default limit is 20 instances.

For information about requesting limit increases for AWS resources, see AWS Service Limits.

Best Practices for Hyperparameter Tuning

Hyperparameter optimization is not a fully-automated process. To improve optimization, use the following guidelines when you create hyperparameters.

Topics

- Choosing the Number of Hyperparameters (p. 308)
- Choosing Hyperparameter Ranges (p. 308)
- Using Logarithmic Scales for Hyperparameters (p. 308)
- Choosing the Best Number of Concurrent Training Jobs (p. 308)
- Running Training Jobs on Multiple Instances (p. 308)

Choosing the Number of Hyperparameters

The difficulty of a hyperparameter tuning job depends primarily on the number of hyperparameters that Amazon SageMaker has to search. Although you can simultaneously use up to 20 variables in a hyperparameter tuning job, limiting your search to a much smaller number is likely to give better results.

Choosing Hyperparameter Ranges

The range of values for hyperparameters that you choose to search can significantly affect the success of hyperparameter optimization. Although you might want to specify a very large range that covers every possible value for a hyperparameter, you will get better results by limiting your search to a small range of values. If you get the best metric values within a part of a range, consider limiting the range to that part.

Using Logarithmic Scales for Hyperparameters

During hyperparameter tuning, Amazon SageMaker attempts to figure out if your hyperparameters are log-scaled or linear-scaled. Initially, it assumes that hyperparameters are linear-scaled. If they should be log-scaled, it might take some time for Amazon SageMaker to discover that. If you know that a hyperparameter should be log-scaled and can convert it yourself, doing so could improve hyperparameter optimization.

Choosing the Best Number of Concurrent Training Jobs

Running more hyperparameter tuning jobs concurrently gets more work done quickly, but a tuning job improves only through successive rounds of experiments. Typically, running one training job at a time achieves the best results with the least amount of compute time.

Running Training Jobs on Multiple Instances

When a training job runs on multiple instances, hyperparameter tuning uses the last-reported objective metric from all instances of that training job as the value of the objective metric for that training job. Design distributed training jobs so that you get they report the objective metric that you want.

Provide Dataset Metadata to Training Jobs with an Augmented Manifest File

To classify data into different groupings, you train a model by using a dataset and metadata that act as labels. To include metadata with your dataset in a training job, use an augmented manifest file.

When using an augmented manifest file, your dataset must be stored in Amazon Simple Storage Service (Amazon S3) and you must configure your training job to use dataset stored there. You specify the location and format of this dataset for one or more Channel (p. 876).

When specifying a channel's parameters, you specify a path to the file, called a `S3Uri`. Amazon SageMaker interprets this URI based on the specified `S3DataType` in S3DataSource (p. 994). The `AugmentedManifestFile` option defines a manifest format that includes metadata with the input data. Using an augmented manifest file is an alternative to preprocessing when you have labeled data. For training jobs using labeled data, you typically need to preprocess the dataset to combine input data with metadata before training. If your training dataset is large, preprocessing can be time consuming and expensive.

Augmented Manifest File Format

An augmented manifest file must be formatted in JSON Lines format. In JSON Lines format, each line in the file is a complete JSON object followed by a newline separator.

During training, Amazon SageMaker parses each JSON line and sends some or all of its attributes on to the training algorithm. You specify which attribute contents to pass and the order in which to pass them with the `AttributeNames` parameter of the CreateTrainingJob (p. 667) API. The `AttributeNames` parameter is an ordered list of attribute names that Amazon SageMaker looks for in the JSON object to use as training input.

For example, if you list `["line", "book"]` for `AttributeNames`, the input data must include the attribute names of `line` and `book` in the specified order. For this example, the following augmented manifest file content is valid:

```
{"author": "Herman Melville", "line": "Call me Ishmael", "book": "Moby Dick"}
{"line": "It was love at first sight.", "author": "Joseph Heller", "book": "Catch-22"}
```

Amazon SageMaker ignores unlisted attribute names even if they precede, follow, or are in between listed attributes.

When using augmented manifest files, observe the following guidelines:

- The order of the attributes listed in the `AttributeNames` parameter determines the order of the attributes passed to the algorithm in the training job.
- The listed `AttributeNames` can be a subset of all of the attributes in the JSON line. Amazon SageMaker ignores unlisted attributes in the file.
- You can specify any type of data allowed by the JSON format in `AttributeNames`, including text, numerical, data arrays, or objects.
- To include an S3 URI as an attribute name, add the suffix `-ref` to it.

If an attribute name contains the suffix `-ref`, the attribute's value must be an S3 URI to a data file that is accessible to the training job. For example, if `AttributeNames` contains `["image-ref", "is-a-cat"]`, a valid augmented manifest file might contain these lines:

```
{"image-ref": "s3://mybucket/sample01/image1.jpg", "is-a-cat": 1}
{"image-ref": "s3://mybucket/sample02/image2.jpg", "is-a-cat": 0}
```

For the first line of this manifest, Amazon SageMaker retrieves the contents of the S3 object `s3://mybucket/foo/image1.jpg` and streams it to the algorithm for training. The second line is the string representation of the `is-a-cat` attribute `"1"`, which is followed by the contents of the second line.

To create an augmented manifest file, use Amazon SageMaker Ground Truth to create a labeling job. For more information, see Output Data (p. 545).

Stream Augmented Manifest File Data

Augmented manifest files are supported only for channels using Pipe input mode. For each channel, the data is extracted from its augmented manifest file and streamed (in order) to the algorithm through the channel's named pipe. Pipe mode uses the first in first out (FIFO) method, so records are processed in the order in which they are queued. For information about Pipe input mode, see InputMode.

Attribute names with a `"-ref"` suffix point to preformatted binary data. In some cases, the algorithm knows how to parse the data. In other cases, you might need to wrap the data so that records are delimited for the algorithm. If the algorithm is compatible with RecordIO-formatted data, specifying `RecordIO` for `RecordWrapperType` solves this issue. If the algorithm is not compatible with `RecordIO` format, specify `None` for `RecordWrapperType` and make sure that your data is parsed correctly for your algorithm. Using the `["image-ref", "is-a-cat"]` example, if you use RecordIO wrapping, the following stream of data is sent to the queue:

```
recordio_formatted(s3://mybucket/foo/
image1.jpg)recordio_formatted("1")recordio_formatted(s3://mybucket/bar/
image2.jpg)recordio_formatted("0")
```

Images that aren't wrapped with RecordIO format, are streamed with the corresponding `is-a-cat` attribute value as one record. This can cause a problem because the algorithm might not delimit the images and attributes correctly.

With augmented manifest files and Pipe mode in general, size limits of the EBS volume do not apply. This includes settings that otherwise must be within the EBS volume size limit such as S3DataDistributionType. For more information about Pipe mode and how to use it, see Using Your Own Training Algorithms - Input Data Configuration.

Use an Augmented Manifest File (Console)

To complete this procedure, you need:

- The URL of the S3 bucket where you've stored the augmented manifest file.
- To store the data that is listed in the augmented manifest file in an S3 bucket.
- The URL of the S3 bucket where you want to store the output of the job.

To use an augmented manifest file in a training job (console)

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker.
2. In the navigation pane, choose **Training**, then choose **Training jobs**.
3. Choose **Create training job**.
4. Provide a name for the training job. The name must be unique within an AWS Region in an AWS account. It can have 1 to 63 characters. Valid characters: a-z, A-Z, 0-9, and . : + = @ _ % - (hyphen).
5. Choose the algorithm that you want to use. For information about supported built-in algorithms, see Use Amazon SageMaker Built-in Algorithms (p. 56). If you want to use a custom algorithm, make sure that it is compatible with Pipe mode.
6. (Optional) For **Resource configuration**, either accept the default values or, to reduce computation time, increase the resource consumption.

   a. (Optional) For **Instance type**, choose the ML compute instance type that you want to use. In most cases, **ml.m4.xlarge** is sufficient.

   b. For **Instance count**, use the default, `1`.

   c. (Optional) For **Additional volume per instance (GB)**, choose the size of the ML storage volume that you want to provision. In most cases, you can use the default, `1`. If you are using a large dataset, use a larger size.

7. Provide information about the input data for the training dataset.
   a. For **Channel name**, either accept the default (**`train`**) or enter a more meaningful name, such as **`training-augmented-manifest-file`**.
   b. For **InputMode**, choose **Pipe**.
   c. For **S3 data distribution type**, choose **FullyReplicated**. When training incrementally, fully replicating causes each ML compute instance to use a complete copy of the expanded dataset. For neural-based algorithms, such as Neural Topic Model (NTM) Algorithm (p. 177), choose `ShardedByS3Key`.
   d. If the data specified in the augmented manifest file is uncompressed, set the **Compression type** to **None**. If the data is compressed using gzip, set it to **Gzip**.
   e. (Optional) For **Content type**, specify the appropriate MIME type. Content type is the multipurpose internet mail extension (MIME) type of the data.
   f. For **Record wrapper**, if the dataset specified in the augmented manifest file is saved in RecordIO format, choose **RecordIO**. If your dataset is not saved as a RecordIO-formatted file, choose **None**.
   g. For **S3 data type**, choose **AugmentedManifestFile**.
   h. For **S3 location**, provide the path to the bucket where you stored the augmented manifest file.
   i. For **AugmentedManifestFile attribute names**, specify the name of an attribute that you want to use. The attribute name must be present within the augmented manifest file, and is case-sensitive.
   j. (Optional) To add more attribute names, choose **Add row** and specify another attribute name for each attribute.
   k. (Optional) To adjust the order of attribute names, choose the up or down buttons next to the names. When using an augmented manifest file, the order of the specified attribute names is important.
   l. Choose **Done**.
8. For **Output data configuration**, provide the following information:
   a. For **S3 location**, type the path to the S3 bucket where you want to store the output data.
   b. (Optional) You can use your AWS Key Management Service (AWS KMS) encryption key to encrypt the output data at rest. For **Encryption key**, provide the key ID or its Amazon Resource Number (ARN). For more information, see KMS-Managed Encryption Keys.
9. (Optional) For **Tags**, add one or more tags to the training job. A *tag* is metadata that you can define and assign to AWS resources. In this case, you can use tags to help you manage your training jobs. A tag consists of a key and a value, which you define. For example, you might want to create a tag with **`Project`** as a key and a value that refers to a project that is related to the training job, such as **`Home value forecasts`**.
10. Choose **Create training job**. Amazon SageMaker creates and runs the training job.

After the training job has finished, Amazon SageMaker stores the model artifacts in the bucket whose path you provided for **S3 output path** in the **Output data configuration** field. To deploy the model to get predictions, see Step 6: Deploy the Model to Amazon SageMaker (p. 26).

Use an Augmented Manifest File (API)

The following shows how to train a model with an augmented manifest file using the Amazon SageMaker high-level Python library:

```
# Create a model object set to using "Pipe" mode.
model = sagemaker.estimator.Estimator(training_image,
                                      role,
                                      train_instance_count=1,
```

```
                                        train_instance_type='ml.p3.2xlarge',
                                        train_volume_size = 50,
                                        train_max_run = 360000,
                                        input_mode = 'Pipe',
                                        output_path=s3_output_location,
                                        sagemaker_session=session)

# Create a train data channel with S3_data_type as 'AugmentedManifestFile' and attribute
 names.
train_data = sagemaker.session.s3_input(your_augmented_manifest_file,
                                         distribution='FullyReplicated',
                                         content_type='image/jpeg',
                                         s3_data_type='AugmentedManifestFile',
                                         attribute_names=['source-ref', 'annotations'])
data_channels = {'train': train_data}

# Train a model.
model.fit(inputs=data_channels, logs=True)
```

After the training job has finished, Amazon SageMaker stores the model artifacts in the bucket whose path you provided for **S3 output path** in the **Output data configuration** field. To deploy the model to get predictions, see Step 6: Deploy the Model to Amazon SageMaker (p. 26).